

Software Quality Engineering

Software errors, faults, and failures

Software Error

- An error is a human mistake made during software development (e.g., coding, design, or requirement specification).
- It is an incorrect action or decision by the developer that may lead to defects in the software.

Software Fault (Defect or Bug)

- A fault is a flaw or defect in the software code, design, or documentation that results from an error.
- It exists in the software but may not always cause immediate problems.

Software Failure

- A failure occurs when a fault is triggered during execution, causing the software to behave unexpectedly or incorrectly.
- It is the observable result of a fault affecting system behavior.

Difference Between Software Quality Assurance (SQA) and Software Quality Engineering (SQE)

Aspect	Software Quality Assurance (SQA)	Software Quality Engineering (SQE)
Definition	A process-focused approach ensuring that software development follows predefined quality standards and processes.	An engineering discipline that applies systematic techniques to improve software quality through testing, automation, and defect prevention.
Focus	Process-oriented (ensures proper methodologies are followed).	Product-oriented (ensures software meets quality requirements).
Scope	Covers the entire software development lifecycle (SDLC) to prevent defects.	Primarily focuses on testing, validation, and defect detection.
Methods Used	Audits, process monitoring, compliance checks, and quality reviews.	Automated testing, performance analysis, defect prevention techniques, and coding best practices.
Preventive vs Corrective	Preventive – aims to avoid defects before they occur.	Corrective – focuses on identifying and fixing defects in software.
Examples of Activities	Process improvement, standards compliance, documentation, training, and quality audits.	Test case design, code reviews, test automation, debugging, and performance tuning.

Causes of software errors

Frame 1.7: The nine causes of software errors

The nine causes of software errors are:

- a. Faulty requirements definition
- b. Client–developer communication failures
- c. Deliberate deviations from software requirements
- d. Logical design errors
- e. Coding errors
- f. Noncompliance with documentation and coding instructions
- g. Shortcomings of the testing process
- h. User interface and procedure errors
- i. Documentation errors

Additional approaches to classification of software defects and their causes are presented by Ko and Myers (2005) and Thung et al. (2012).

McCall's classic model for software quality factors

- Several models of software quality factors and their classification to factor categories have been suggested over the years.
- McCall's factor model classifies all software requirements into 11 software quality factors.
- The **purpose of McCall's Software Quality Factor Model** is to define and evaluate software quality by breaking it into measurable factors. It helps software engineers, developers, and quality assurance teams assess software based on key attributes that ensure its effectiveness, maintainability, and usability.

Key Purposes of McCall's Model:

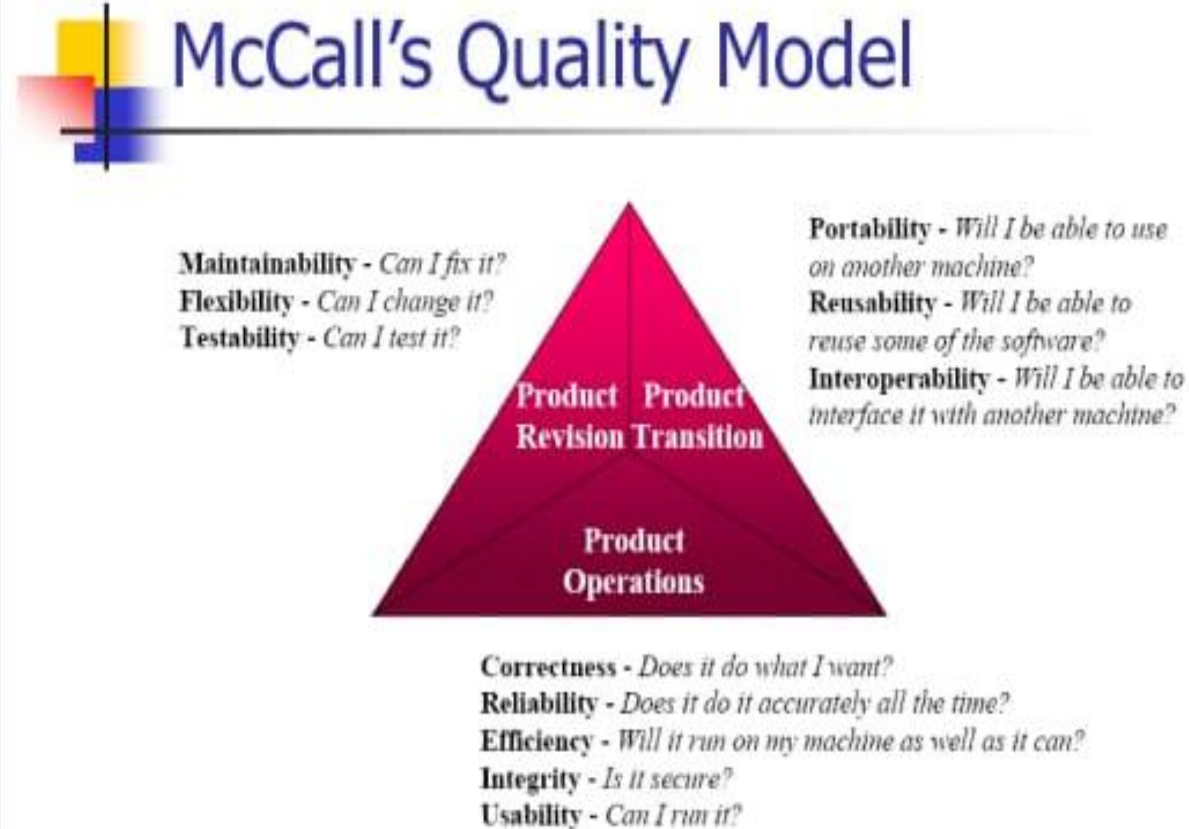
1. Defines Software Quality – It provides a structured way to understand what makes software high quality.
2. Guides Software Development – Helps developers focus on key aspects like correctness, reliability, and maintainability.
3. Improves Maintainability – Ensures software can be easily modified, tested, and updated.
4. Supports Decision-Making – Assists managers and engineers in making informed decisions about software design, development, and quality control.
5. Ensures Usability & Performance – Emphasizes user-friendliness, efficiency, and resource optimization.
6. Aids in Software Evaluation – Provides a framework for testing and quality assurance teams to assess software products.

McCall Software Quality Model

It was introduced by **James A. McCall** in the 1970s and focuses on software from three perspectives:

1. **Product Revision (Maintaining Software)**
2. **Product Transition (Adapting to New Environments)**
3. **Product Operations (Functionality in Use)**

McCall's model



Product Revision

McCall's Quality Model (Cont.)

Product Revision	•Maintainability	◦Consistency ◦Simplicity ◦Conciseness ◦Self-descriptiveness ◦Modularity
	•Testability	◦Simplicity ◦Scope for Automation
	•Flexibility	◦Expandability ◦Architecture Usability ◦Self descriptiveness ◦Modularity

Product transition

McCall's Quality Model (Cont.)

Product Transition	•Re-usability	○Architectural Usability ○Self Descriptiveness ○Modularity ○Machine Independence ○Operating system Independence
	•Portability	○Self Descriptiveness ○Modularity ○Machine Independence ○OS Independence

Product Operations

McCall's Quality Model (Cont.)

Use	Factor	Criteria
Product Operation	•Usability	○Operability ○Training ○Communicativeness ○I/O Volume ○I/O Rate
	•Integrity	○Access Control ○Access Audit
	•Efficiency	○Storage Requirement ○Execution Speed
	•Correctness	○Traceability ○Completeness ○Consistency
	•Reliability	○Accuracy ○Error Tolerance ○Consistency ○Simplicity

Product Revision (Maintaining Software)

- Maintainability – How easy it is to modify the software to correct defects, improve performance, or adapt to changes.
- Flexibility – The ability of software to be changed without major rework.
- Testability – The ease with which software can be tested to ensure it works correctly.

Product Transition (Adapting to New Environments)

- Portability – The ability of software to work on different hardware and operating systems with minimal changes.
- Reusability – How much of the software's components can be reused in other applications.
- Interoperability – The ability of software to communicate and operate with other systems.

Product Operations (Functionality in Use)

- Correctness – The degree to which software meets specified requirements and performs as expected.
- Reliability – How consistently the software performs without failures over time.
- Efficiency – The software's ability to use system resources optimally (CPU, memory, etc.).
- Integrity – The ability to prevent unauthorized access or data corruption.
- Usability – How user-friendly and easy to operate the software is.